QALight

# Frontend-розробка

Заняття 17. Події

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

# Порядок денний



Події.
Керування подіями. Обробники подій. Обробка подій у DOM. Слухачі подій. Об'єкт Event. Фази поширення події. Спливання подій. Захоплення подій. Делегування подій.
Створення власної події.

# Події

Керування подіями. Обробники подій. Обробка подій у DOM. Слухачі подій. Об'єкт Event. Фази поширення події. Спливання подій. Захоплення подій. Делегування подій.

# Події JavaScript

Теорія: Events

Будь-яка дія (системна чи створена користувачем), на яку ви можете відреагувати, називається **подією**. Те, як ми реагуємо на це, називається **обробкою подій**.

На веб-сторінці відбувається маса подій:

- Користувач наводить курсор на елемент
- Надсилається форма
- Відтворення відео завершено
- Користувач прокручує зображення
- Зміна розміру вікна браузера
- Натискання клавіш
- Завантаження документа завершено

Усе це події, на які можна реагувати. Для розробників події допомагають визначити, що щойно сталося в програмі, який стан користувача був у певний час тощо.

# Активація подій

Теорія: Events

**Події** - це сигнали від браузера про те, що на сторінці щось сталося (наприклад, користувач натиснув кнопку, завантажився документ або змінився розмір вікна). **Функції** можуть реагувати на ці сигнали та виконувати певні дії, що можна прив'язати до елементів HTML сторінки. Код подій виконується лише після того, як відбувається їх **активація**. Обробник події - це просто функція, яка запускається, коли відбувається певна подія. Існують різні способи призначення обробників подій у JavaScript:

- можна встановити обробник подій **безпосередньо в коді HTML** за допомогою таких **атрибутів**, як **onclick, onmouseover** тощо.

```
<button onclick="alert('Button clicked!')">Click me</button>
```

- можна призначити обробник за допомогою **властивості DOM**, наприклад onclick, onmouseover тощо.

```
document.getElementById("myButton").onclick = function() {
    alert('Button clicked!');
};
```

**JavaScript - однопотокова мова**, отже **обробники завжди виконуються послідовно та у загальному потоці**. При встановленні обробників двох подій, що виникають на елементі одночасно, наприклад mouseover і mousemove, їх **обробники будуть виконуватись послідовно**.

# Основні категорії подій

Теорія: Events

**Події миші:**

click (клік), dblclick (подвійний клік), mouseover (наведення курсора), mousedown (натискання кнопки миші).

**Події клавіатури:**

keydown (натискання клавіші), keyup (відпускання клавіші).

**Події форми:**

submit (відправка форми), focus (отримання фокуса елементом), blur (втрата фокуса), change (зміна значення в полі введення).

**Події документа та вікна:**

DOMContentLoaded (HTML повністю завантажено), load (повне завантаження сторінки і всіх ресурсів), resize (зміна розміру вікна).

# Зворотні виклики

Теорія: Events

Функція зворотного виклику - це функція, яка передається іншій функції як аргумент. Потім функцію зворотного виклику можна викликати всередині викликаної функції, щоб виконати певну дію.

Використовуючи зворотні виклики, ми можемо вказати, що певний блок коду повинен бути запущений у відповідь на певну подію:

```
function handleClick() {
    // Do something in response to the user clicking a button
}
document.querySelector('button').addEventListener('click', handleClick);
```

Функція handleClick є зворотним викликом, який виконується у відповідь на дію, що відбувається на веб-сторінці. Використовуючи цей підхід, ми можемо реагувати на скільки завгодно подій, залишаючи інтерпретатор JavaScript вільним для виконання будь-яких інших завдань.

# Слухачі подій

Теорія: Events

**Слухачі подій** - це функції, які очікують настання певної події, а потім виконують призначену функцію. Зазвичай це робиться за допомогою методу **addEventListener**.

Прослуховувачі подій і обробники подій **взаємозамінні**, однак є невелика технічна відмінність. Слухачі слухають, коли відбувається подія, а обробником є код, який виконується.

Щоразу, коли ми приєднуємо обробник події до події, ми називаємо це **реєстрацією обробника події**.

```
<button class="btn-primary">Натисніть мене!</button>
const myButton = document.querySelector(".btn-primary");
myButton.addEventListener("click", function() {
   console.log("The button was clicked");
});
```

# Метод addEventListener

Теорія: Events

Метод **addEventListener** приймає 3 аргументи target.addEventListener(type, listener[, options]);

- **type -** рядок, що задає тип події.
- **listener -** об’єкт, що отримує повідомлення - є **функція зворотного виклику,** що виконується після настання події.
- **options (**необов’язковий**) -** об’єкт, що визначає характеристики об’єкта.

Метод target.**addEventListener() реєструє обробник події** на об’єкті target. target має бути існуючим елементом у документі, або **document**, або **window,** або іншим об’єктом, шо **підтримує події** (XMLHttpRequest).

**Метод зворотного виклику**, наданий слухачу подій, є **анонімною функцією**. Функція не має назви, і на неї неможливо посилатися з будь-якого місця. Метод може бути іменованою функцією.

```
const handleClick = function() {
    console.log("The button was clicked");
};
myButton.addEventListener("click", handleClick);
```

# Події документа

Теорія: Events

Події документа пов’язані з самим об’єктом document. Процес завантаження документа має 3 стадії:

1. **DOMContentLoaded** – браузер повністю завантажив HTML та збудував DOM-дерево.
2. **load** – браузер завантажив всі ресурси.
3. **beforeunload/unload** – вихід зі сторінки.

Обробник події **DOMContentLoaded** призначається **лише** через **addEventListener**:

```
document.addEventListener("DOMContentLoaded", messageShow);
```

Подія window.**onload** відбувається, коли завантажується вся сторінка, включаючи стилі, картинки, фрейми тощо. Подія load **використовується рідко**.

```
window.onload = function() {  messageShow();   };
```

Коли користувач йде зі сторінки або закриває вікно, відбувається подія **window.onunload**. Для відміни переходу можна використати подію **Window.onbeforeunload:**

```
window.onbeforeunload = function() {return "This page is asking you to confirm that you want to leave"; };
```

Більшість браузерів показують стандартне повідомлення: **This page is asking you to confirm that you want to leave - data you have entered may not be saved.**

# Властивість document.readyState

Теорія: Events

Властивість document.**readyState** описує стан завантаження документа. Коли значення цієї властивості змінюється, для об’єкта документа запускається подія **readystatechange**.

Стан документа readyState документа може бути одним із таких:

- **loading**: Документ ще завантажується.
- **Interactive:** Документ було проаналізовано, але підресурси, такі як сценарії, зображення, таблиці стилів і фрейми, все ще завантажуються. Стан вказує на те, що подія DOMContentLoaded збирається запуститися.
- **complete**: Документ і всі підресурси завантажено.

DOMContentLoaded може спрацювати до того, як сценарій встигне запуститися, тому доцільно перевірити, перш ніж додавати слухача.

```
if (document.readyState === "loading") {
  document.addEventListener("DOMContentLoaded", main);
} else {
  main();
}
```

# Подія DOMContentLoaded

Теорія: Events

```
// Alternative to DOMContentLoaded event
document.onreadystatechange = () => {
 if (document.readyState === "interactive") {
   initApplication();
 }};
// readystatechange як альтернатива події завантаження
document.onreadystatechange = () => {
 if (document.readyState === "complete") {
   initApplication();
 }};
// readystatechange як слухач подій, щоб вставити або змінити DOM перед DOMContentLoaded
document.addEventListener("readystatechange", (event) => {
 if (event.target.readyState === "interactive") {
   initLoader();
 } else if (event.target.readyState === "complete") { initApp(); }
});
```

# Успадкування значення this

Теорія: Events

Анонімні функції добре працюють у коротких сценаріях, де функція завжди викликається лише в одному місці, і оскільки вони вбудовані, також мають доступ до батьківської області.

```
document.querySelector('form').addEventListener('submit', function(e)  {
        e.preventDefault();
        // Do some data validation
        // If everything looks ok, then...
        this.submit();
});
```

Функції стрілок часто використовують для виконання простих однорядкових команд, як, наприклад, у наступному прикладі, який фільтрує дублі значень з масиву:

```
const arr = [1, 2, 2, 3, 4, 5, 5];
const unique = arr.filter((el, i) => arr.indexOf(el) === i);
// [1, 2, 3, 4, 5]
```

Функції стрілок успадковують значення this від батьківської області.

# Події, що пов’язані з мишею

Теорія: Events

Події пов’язані з мишею:
ґonclick, ondblclick, mouseover / mouseout, mousedown / mouseup, onmousemove

```
// Mouse Button Property Example:
<div onmousedown="console.log('Mouse button:', event.button)">Click me</div>

// Modifier Keys Example:
<div onclick="if(event.ctrlKey) console.log('Ctrl + Click!')">Ctrl + Click Me</div>

// Coordinates Example:
<div onmousemove="console.log('clientX:', event.clientX, 'clientY:', event.clientY)">Move your mouse here</div>ґ
ї
// Preventing Selection Example:
<span ondblclick="console.log('Double clicked!')" onmousedown="return false;">Double-click me</span>
```

# Події на клавіатурі

Теорія: Events

Події на клавіатурі – натискання клавіші/вгору: коли натискається або відпускається клавіша на клавіатурі. Наприклад цей код очікує натискання клавіші Enter, а потім змінює вигляд і вміст елемента сповіщення на сторінці:

```
btnInfo.addEventListener("keypress", function(event) {
 if (event.key === "Enter") {
   event.preventDefault();
   alert.classList.add('alert-info');
   alert.textContent = "A simple info alert—check it out!";
 }
});
```

Подія, яка прослуховується, є подією натискання клавіші. Коли користувач натискає клавішу на клавіатурі, виконується функція, зазначена в коді. Усередині функції є умовний оператор, який перевіряє, чи була натиснута клавіша клавішею "Enter". Якщо це так, типова дія події скасовується за допомогою методу event.preventDefault(). Після скасування дії за замовчуванням код додає клас CSS "alert-info" до елемента з класом "alert". Він також встановлює текстовий вміст цього елемента A simple info alert—check it out!

# Метод removeEventListener()

Теорія: Events

Метод **removeEventListener()** використовується для видалення попередньо приєднаного слухача подій для елемента. Для видалення прослуховувач подій, потрібно передати два аргументи подію та метод обробки подій. Неможливо вказати другий параметр, якщо зворотний виклик був анонімною функцією.

```
function handleClick() {
    alert('Button clicked!');
}
document.getElementById("myButton").addEventListener('click', handleClick);

// Remove the event handler
document.getElementById("myButton").removeEventListener('click', handleClick);
```

# Об’єкт event

Теорія: Events

Коли відбувається подія, браузер створює об’єкт події - **event**, який містить інформацію про подію, наприклад тип події, цільовий елемент і будь-які додаткові дані. Цей об’єкт передається як аргумент функції обробника подій, до якої можна отримати доступ у функції зворотного виклику прослухувача подій.

```
element.addEventListener('click', function(event) {
    console.log(event.type); // Output: "click"
    console.log(event.target); // Output: The element that was clicked
    // We can access more properties like event.clientX, event.clientY, etc.
});
```

# Метод handleEvent

Теорія: Eventst

Замість того, щоб призначати функцію як обробник події, ви також можете призначити об’єкт, який має метод **handleEvent**. Коли відбувається подія, буде викликано метод handleEvent об’єкта.

```
<button id="myButton">Click me</button>
<script>
let myObject = {
    handleEvent: function(event) {
    alert('Button clicked!');
    }
};

document.getElementById("myButton").addEventListener('click', myObject);
</script>
```

# Фази подій

Теорія: Events

Події в DOM можуть поширюватися у 3 фази: **фаза захоплення, цільової фази та фаза спливання**. Коли на елементі відбувається подія, браузеру потрібно вирішити, які елементи мають бути повідомлені про подію.
На етапі **захоплення (capturing)** подія починається з вершини ієрархії DOM (елемент <html>) і йде вниз до цільового елемента. Під час цієї фази запускаються обробники подій, додані за допомогою **addEventListener** і параметр захоплення, встановлений на **true**. Ці обробники виконуються до того, як подія досягне цільового елемента.
Коли подія досягає цільового елемента, вона переходить у **цільову фазу**, де запускаються обробники подій, додані за допомогою **addEventListener без опції захоплення**. Обробники, додані на цьому етапі, виконуються, коли подія спрямована безпосередньо на елемент.
Після цільової фази подія **піднімається** з цільового елемента на вершину ієрархії DOM. Під час цієї фази обробники подій, прикріплені за допомогою addEventListener без опції захоплення (або зі значенням false), запускаються знову. Обробники, додані на цій фазі, виконуються, коли подія переміщується вгору від цільового елемента.

# Спливання подій

Теорія: Events

**Спливання подій** - це механізм у JavaScript, за якого, коли подія відбувається в елементі, ця подія спочатку запускається в цільовому елементі, а потім спливає через елементи-предки аж до кореня документа.

**Властивість event.target** посилається на елемент, який ініціював подію і залишається незмінним протягом усього процесу спливання.

Спливання подій у JavaScript означає, що коли подія відбувається з елементом, викликається його обробник, а потім його батьки, а потім предки.

Подія проходить через кожен батьківський елемент в ієрархії DOM.

Не всі події спливають.

Кілька таких подій, що не спливають: **load, unload, focus, blur**

# Елемент, де відбулась подія, називається цільовим

Теорія: Events

Коли подія проходить через кілька фаз, важко відстежити, що спричинило ланцюг подій. Елемент, на якому відбулась подія,називається **цільовим або event.target**. Незалежно від того, на скільки фазах з'являється подія, **event.target ніколи не змінюється**.

Цільовий елемент - це елемент, який **ініціював подію**. Він залишається незмінним протягом усього процесу обробки події.

Ключове слово **this** або **event.currentTarget** є поточним елементом обробки події. Події, які з'являються під час спливання, потрапляють до елемента <html>.

Якщо не потрібно викликати будь-які обробники з батьківських елементів, використовують **event.stopPropagation()**. Цей метод зупиняє поширення події на батьківські елементи.

```
<button onclick="event.stopPropagation()">Click me</button>
```

# Об’єкт event.target

Теорія: Events

Цей код отримує доступ до властивості **offsetParent** об’єкта **event.target** і реєструє його на консолі. Властивість offsetParent повертає **найближчий елемент предка цільового елемента**.

```
let cardObject = {
 handleEvent: function(event) {
   console.log(event);
   console.log(event.target.offsetParent);
   let parent = event.target.offsetParent;
   console.log(parent.querySelector('.card-title').textContent);
 }
};
for (let i = 0; i < cards.length; i++) {
 let addToCartButton = cards[i].querySelector('.add-to-cart');
 addToCartButton.addEventListener('click', cardObject);
}
```

# Захоплення подій

Теорія: Events

**Захоплення подій** є протилежністю спливання подій. У випадку спливання подій, події випливають з батьківського елемента, а потім його предків.

У разі захоплення подій, захоплення починається зверху, спускаючись вниз по структурі DOM, поки не досягне цільового елемента. Цільовий елемент є загальним для спливання та захоплення. Захоплення закінчується на цілі, а спливання починаються на цілі.

Щоб увімкнути захоплення подій, ми можемо передати третій параметр **addEventListener**.

```
const myButton = document.querySelector(".btn-primary");
myButton.addEventListener("click", function() {
    console.log("The button was clicked");
}, { capture : true });
```

Третій параметр не обов’язково повинен бути об’єктом. Це може бути логічне значення true.

# Делегування події

Теорія: Events

Замість того, щоб приєднувати прослуховувачі подій безпосередньо до елемента, делегують прослуховування батьківському елементу. Слухач батьківського елемента ловить подію.
Для використання делегування подій потрібно виконати 3 кроки:

1. Визначити батьківський елемент для спостереження за подіями
2. Приєднати слухача подій до батьківського елемента
3. Використати event.target, щоб вибрати цільовий елемент

Властивість event.target отримує доступ до елемента, на який було надіслано подію

```
.addEventListener('click', event => {
    if (event.target.className === 'buttonClass') { console.log('Click!'); }
});
```

event.currentTarget вказує на елемент, до якого безпосередньо приєднаний слухач подій.
Властивість **Event.target** інтерфейсу **Event** є **посиланням на об'єкт,** на який було надіслано подію.
Властивість **event.target** можна використовувати для реалізації **делегування події**.

```
let addToCart = modalWindow.querySelector('.add-to-cart');
    addToCart.addEventListener('click', event => {
            let productId = event.target.dataset.id;
            addProductToCart({id:productId}, +quantityResult.value);
    });
```

# Метод forEach

Практика: Events

Сучасний NodeList має вбудований метод forEach, який працює точно так само, як і для звичайних масивів. Це найбільш лаконічний та читабельний варіант.

```
const links = document.querySelectorAll('.nav-link');
// Перебираємо кожен елемент списку
links.forEach(function(link) {
   link.style.color = 'red';
});
// Або коротший запис через стрілочну функцію (Arrow Function):
links.forEach(link => link.style.color = 'red');
```

Виклик події

Створення інтерактивних вкладок

# Створення інтерактивних вкладок

Практика: Events

Створимо три кнопки перемикання та три блоки з контентом. За замовчуванням активною буде перша вкладка (додаємо клас active).

```
<div class="tabs-container">
    <!-- Кнопки керування вкладками -->
    <div class="tabs-buttons">
        <button class="tab-btn active" data-tab="tab-1">Програмування</button>
        <button class="tab-btn" data-tab="tab-2">Дизайн</button>
        <button class="tab-btn" data-tab="tab-3">Маркетинг</button>
    </div>
    <!-- Контент вкладок -->
    <div class="tabs-content">
        <div id="tab-1" class="tab-content-item active">
            <h2>Курси Програмування</h2>
            <p>Вивчайте JavaScript, Python та побудову сучасних вебдодатків.</p>
        </div>
        <div id="tab-2" class="tab-content-item">
            <h2>Курси Дизайну</h2>
            <p>Опануйте UI/UX дизайн, роботу з Figma та створення інтерфейсів.</p>
        </div>
        <div id="tab-3" class="tab-content-item">
            <h2>Курси Маркетингу</h2>
            <p>Навчіться налаштовувати таргетинг, SEO та просувати бренди.</p>
        </div>
    </div>
</div>
```

# Приховати неактивні блоки й показати активні

Практика: Events

```
/* Приховуємо всі блоки з контентом за замовчуванням */
    .tab-content-item {
        display: none;
    }

    /* Показуємо лише той блок, який має клас active */
    .tab-content-item.active {
        display: block;
    }

    /* Стиль для активної кнопки, щоб користувач бачив, де він перебуває */
    .tab-btn.active {
        background-color: #007bff;
        color: white;
        font-weight: bold;
    }
```

# Перебираємо список кнопок через цикл forEach

Практика: Events

```
// 1. Знаходимо всі кнопки та всі блоки з контентом
// Використання querySelectorAll:
// Ми знайшли всі потрібні елементи двома короткими рядками.
const tabButtons = document.querySelectorAll('.tab-btn');
const tabContents = document.querySelectorAll('.tab-content-item');


tabButtons.forEach(button => {

    // 2. Перебираємо список кнопок через цикл forEach

});
```

# Вішаємо обробник події (клік) на кожну окрему кнопку

Практика: Events

```
// 2. Перебираємо список кнопок через цикл forEach

tabButtons.forEach(button => {

    // Вішаємо обробник події (клік) на кожну окрему кнопку
    button.addEventListener('click', () => {

        // КРОК А: Очищення. Видаляємо клас active з усіх кнопок
        tabButtons.forEach(btn => btn.classList.remove('active'));

        // КРОК Б: Очищення. Видаляємо клас active з усіх блоків контенту
        tabContents.forEach(content => content.classList.remove('active'));
```

# Вкладений перебір

Практика: Events

Замість того, щоб запам'ятовувати, яка вкладка була відкрита раніше, ми використовуємо швидкий цикл «скидання» (**remove('active')**) перед активацією нової.

```
// 2. Перебираємо список кнопок через цикл forEach

tabButtons.forEach(button => {

    // Вішаємо обробник події (клік) на кожну окрему кнопку
    button.addEventListener('click', () => {

        // КРОК А: Очищення. Видаляємо клас active з усіх кнопок
        tabButtons.forEach(btn => btn.classList.remove('active'));

        // КРОК Б: Очищення. Видаляємо клас active з усіх блоків контенту
        tabContents.forEach(content => content.classList.remove('active'));
```

# Активація кнопки

Практика: Events

```
// 2. Перебираємо список кнопок через цикл forEach

tabButtons.forEach(button => {

    // Вішаємо обробник події (клік) на кожну окрему кнопку
    button.addEventListener('click', () => {

        // КРОК А: Очищення. Видаляємо клас active з усіх кнопок
        tabButtons.forEach(btn => btn.classList.remove('active'));

        // КРОК Б: Очищення. Видаляємо клас active з усіх блоків контенту
        tabContents.forEach(content => content.classList.remove('active'));

        // КРОК В: Активація кнопки. Додаємо клас active кнопці, на яку натиснули
        button.classList.add('active');
```

# Data-атрибути

Практика: Events

Data-атрибути (наприклад, data-tab) — це спеціальний стандартний механізм у HTML5, який дозволяє зберігати додаткову інформацію безпосередньо в HTML-тегах. Це ідеальний інструмент для зв'язку розмітки з JavaScript, особливо при створенні інтерактивних компонентів на кшталт вкладок (tabs), перемикачів або галерей.
Усі такі атрибути обов'язково повинні починатися з префікса data-.

Додамо атрибут data-tab, щоб пов'язати кнопку перемикання з відповідним блоком контенту:

```
<!-- Кнопки керування вкладками -->
    <div class="tabs-buttons">
        <button class="tab-btn active" data-tab="tab-1">Програмування</button>
        <button class="tab-btn" data-tab="tab-2">Дизайн</button>
        <button class="tab-btn" data-tab="tab-3">Маркетинг</button>
    </div>
```

# Як зчитувати та змінювати data- атрибути

Практика: Events

В JavaScript для роботи з атрибутами data- є спеціальна властивість **dataset**. Браузер автоматично збирає всі data- атрибути елемента в цей об'єкт, відкидаючи префікс data-.

```
const button = document.querySelector('.tab-btn');
// 1. Зчитування значення (data-tab перетворюється на властивість tab)
console.log(button.営業dataset.tab); // Виведе: "profile"
// 2. Зміна значення в процесі роботи програми
button.dataset.tab = "dashboard"; // Тепер в HTML це виглядатиме як data-tab="dashboard"
// 3. Додавання нового data-атрибуту через JS
button.dataset.status = "active"; // Додасть атрибут data-status="active"
```

Якщо назва атрибуту в HTML містить дефіси (наприклад, data-tab-id="15"), в JavaScript вона автоматично перетвориться на camelCase: element.dataset.tabId.

# Активація контенту

Практика: Events

```
tabButtons.forEach(button => {
    // Вішаємо обробник події (клік) на кожну окрему кнопку
    button.addEventListener('click', () => {
        // КРОК А: Очищення. Видаляємо клас active з усіх кнопок
        tabButtons.forEach(btn => btn.classList.remove('active'));
        // КРОК Б: Очищення. Видаляємо клас active з усіх блоків контенту
        tabContents.forEach(content => content.classList.remove('active'));
        // КРОК В: Активація кнопки. Додаємо клас active кнопці, на яку натиснули
        button.classList.add('active');

        // КРОК Г: Активація контенту.
        // Зчитуємо значення атрибута data-tab (наприклад, "tab-2")
        const targetTabId = button.getAttribute('data-tab');
        // Знаходимо блок із відповідним ID та додаємо йому active
        const targetContent = document.getElementById(targetTabId);
        targetContent.classList.add('active');
    });
});
```

# ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

# Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

# Дякую за увагу!

# До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua